Session-A

S1. You are designing the database for a food delivery application. It stores restaurant details, menu items linked to each restaurant, and customer orders. You must decide which columns need NOT NULL constraints, which serve as PRIMARY KEYs, and which link tables via FOREIGN KEYs.

Answer:

1.restaurant_id – PRIMARY KEY:

It uniquely identifies each restaurant. Without a primary key, duplicate restaurant records could exist and it would be difficult to identify a particular restaurant.

2.name – NOT NULL:

A restaurant must have a name. Without NOT NULL, a restaurant record could be inserted without a name, causing incomplete data.

3.restaurant_id in menu_items – FOREIGN KEY:

It links each menu item to an existing restaurant. Without a foreign key, menu items could refer to restaurants that do not exist, creating invalid or orphan records.

Note : PRIMARY KEY → uniquely identifies a record

NOT NULL → prevents missing required data

FOREIGN KEY → maintains relationships between tables

S2. You are managing the database for a food delivery platform. The operations team asks you to update the status of all orders placed more than 7 days ago that are still marked 'Pending' to 'Cancelled', and to permanently delete menu items marked 'Unavailable' from restaurants that have closed.

Answer:

=> The WHERE clause is essential because it specifies which records should be affected by an UPDATE or DELETE statement.

=> For the UPDATE, WHERE ensures that only orders older than 7 days and having status 'Pending' are changed to 'Cancelled'.

=> For the DELETE, WHERE ensures that only menu items marked 'Unavailable' from closed restaurants are deleted.

=> If UPDATE or DELETE is executed without a proper WHERE condition, all records in the table may be updated or deleted. This can cause major data loss or incorrect data and may be difficult to recover.

Simple example :

```
UPDATE orders
SET status = 'Cancelled'
WHERE order_date < CURRENT_DATE - INTERVAL 7 DAY
  AND status = 'Pending';

DELETE FROM menu_items
WHERE status = 'Unavailable'
AND restaurant_id IN (SELECT restaurant_id FROM restaurants WHERE status = 'Closed');
```

Note : WHERE = controls which rows are affected. Without it, SQL cheerfully ruins everyone's afternoon.

S3. You are building a revenue dashboard for a food delivery startup. The management wants total revenue, average order value, and number of orders per restaurant — but only for restaurants that have processed more than 20 orders in the current month.

Answer :

The following SQL clauses and aggregate functions can be used:

- SELECT: to select the required columns and calculated values.

- SUM(): to calculate total revenue.

- AVG(): to calculate the average order value.

- COUNT(): to calculate the number of orders.

- GROUP BY: to group the orders restaurant-wise.

- WHERE: to filter orders belonging to the current month.

- HAVING: to filter restaurants having more than 20 orders.

HAVING must be used instead of WHERE because the condition COUNT(*) > 20 is based on an aggregate function. WHERE filters individual rows before grouping, while HAVING filters groups after GROUP BY.

Example SQL

```
SELECT
   restaurant_id,
   COUNT(*) AS total_orders,
   SUM(total_amount) AS total_revenue,
   AVG(total_amount) AS average_order_value
FROM orders
WHERE MONTH(order_date) = MONTH(CURRENT_DATE)
  AND YEAR(order_date) = YEAR(CURRENT_DATE)
GROUP BY restaurant_id
```

HAVING COUNT(*) > 20;

Note : WHERE → filters rows before GROUP BY

HAVING → filters groups after GROUP BY

S4. You are developing an order history page. The orders table has order_id, customer_id, restaurant_id, and total_amount. The restaurants table has restaurant_id and name. The page must show order ID, restaurant name, and amount for every order — including any that reference a restaurant no longer in the table.

Answer :

INNER JOIN returns only the records that have matching values in both tables.

LEFT JOIN returns all records from the left table and the matching

records from the right table. If there is no match, NULL values are

shown for the right table columns.

For this query, LEFT JOIN should be used because every order must be

displayed, even if its restaurant no longer exists in the restaurants table.

Therefore, the orders table should be the left table and the

restaurants table should be joined using LEFT JOIN.

Example SQL

```
SELECT
   o.order_id,
   r.name AS restaurant_name,
   o.total_amount
FROM orders o
LEFT JOIN restaurants r
   ON o.restaurant_id = r.restaurant_id;
```

S5. You are building the analytics layer for a food delivery platform. A senior developer proposes a VIEW called top_restaurants showing restaurant name, total orders, and average rating — hiding raw ID columns from the front-end team.

Answer :

A VIEW is preferred because it provides a simplified and controlled way for the front-end team to access required data without directly accessing the underlying tables.

It can also hide unnecessary columns such as raw ID columns and improve data security and maintainability.

Two limitations of a VIEW are:

1. A VIEW may have limited ability to perform INSERT, UPDATE, or DELETE operations, especially when it contains GROUP BY or aggregate functions.

2. The VIEW depends on the underlying tables. If the table structure or required columns are changed, the VIEW may stop working or produce incorrect results.

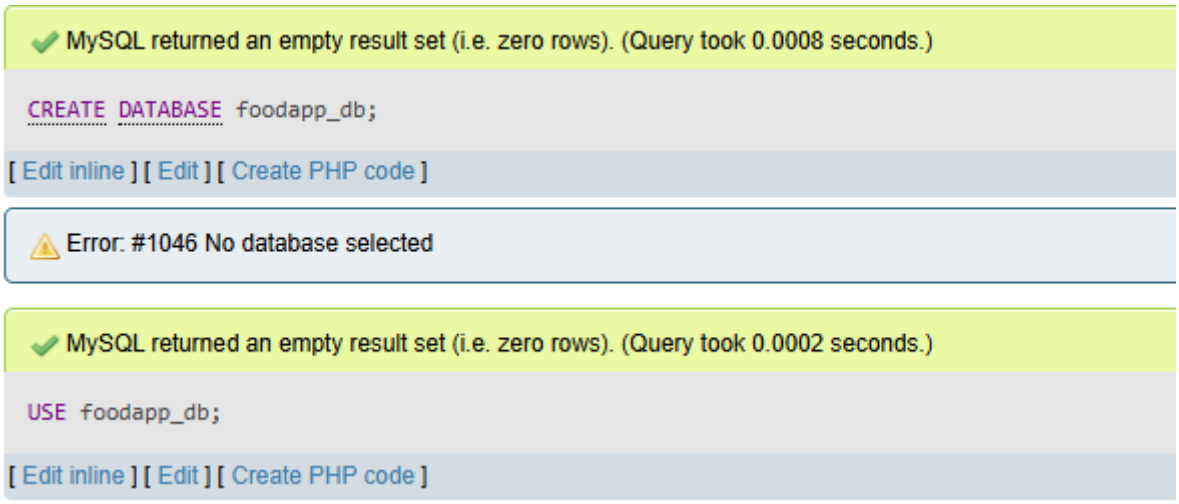
Note : VIEW = a virtual table based on a SQL query, useful for simplifying access and controlling what users see.

SECTION – B

Task 1: Design and Populate a Restaurant Database

CREATE DATABASE foodapp_db;

USE foodapp_db;




```
CREATE TABLE restaurants (
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    city VARCHAR(50),
    cuisine_type VARCHAR(50),
    rating DECIMAL(3,1)
);
```

```
INSERT INTO restaurants (name, city, cuisine_type, rating)
VALUES
('Spice Villa', 'Surat', 'Indian', 4.5),
('Pizza House', 'Ahmedabad', 'Italian', 4.2),
```

('Food Junction', 'Vadodara', 'Chinese', 3.8),

('Green Leaf', 'Surat', 'Gujarati', 4.7),

('Burger Point', 'Ahmedabad', 'Fast Food', 3.5),

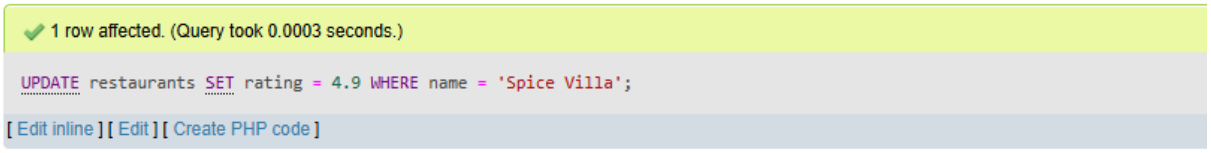
('Royal Kitchen', 'Rajkot', 'North Indian', 4.1);



UPDATE restaurants

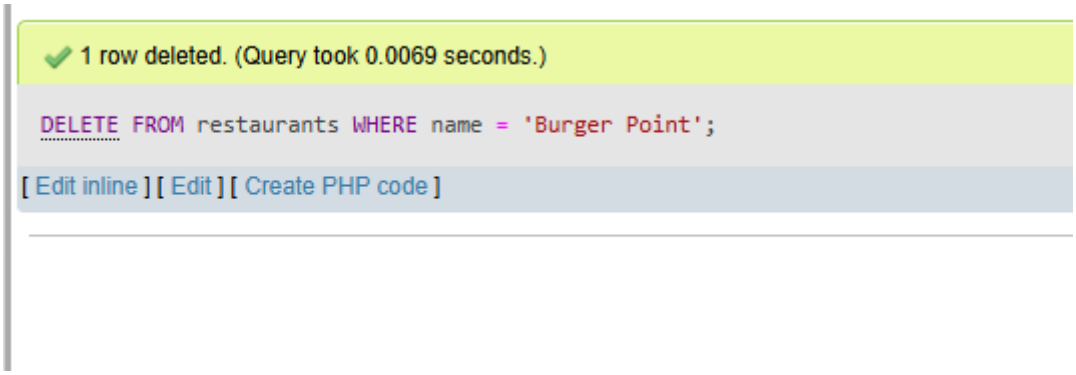
SET rating = 4.9

WHERE name = 'Spice Villa';



DELETE FROM restaurants

WHERE name = 'Burger Point';



SELECT *

FROM restaurants

ORDER BY rating DESC;



	restaurant_id	name	city	cuisine_type	rating
Edit Copy Delete	1	Spice Villa	Surat	Indian	4.9
Edit Copy Delete	4	Green Leaf	Surat	Gujarati	4.7
Edit Copy Delete	2	Pizza House	Ahmedabad	Italian	4.2
Edit Copy Delete	6	Royal Kitchen	Rajkot	North Indian	4.1
Edit Copy Delete	3	Food Junction	Vadodara	Chinese	3.8

Check all *With selected:* Edit Copy Delete Export

Task 2: Aggregate Revenue Reporting on Orders

CREATE TABLE orders (

order_id INT AUTO_INCREMENT PRIMARY KEY,

restaurant_id INT,

customer_name VARCHAR(100),

total_amount DECIMAL(10,2),

order_date DATE

);



INSERT INTO orders

(restaurant_id, customer_name, total_amount, order_date)

VALUES

(1, 'Diya', 1200, '2026-10-01'),

(1, 'Riya', 1800, '2026-10-03'),

(1, 'Amit', 2500, '2026-10-05'),

(2, 'Neha', 2000, '2026-10-02'),

(2, 'Rahul', 1500, '2026-10-04'),

(2, 'Priya', 2200, '2026-10-06'),

(3, 'Karan', 3000, '2026-10-07'),

(3, 'Meera', 2500, '2026-10-08'),

(3, 'Jay', 1000, '2026-10-09'),

(4, 'Krish', 3500, '2026-10-10'),

(4, 'Pooja', 2800, '2026-10-11'),

(4, 'Nisha', 1700, '2026-10-12');

	order_id	restaurant_id	customer_name	total_amount	order_date
Edit Copy Delete	1	1	Diya	1200.00	2026-10-01
Edit Copy Delete	2	1	Riya	1800.00	2026-10-03
Edit Copy Delete	3	1	Amit	2500.00	2026-10-05
Edit Copy Delete	4	2	Neha	2000.00	2026-10-02
Edit Copy Delete	5	2	Rahul	1500.00	2026-10-04
Edit Copy Delete	6	2	Priya	2200.00	2026-10-06
Edit Copy Delete	7	3	Karan	3000.00	2026-10-07
Edit Copy Delete	8	3	Meera	2500.00	2026-10-08
Edit Copy Delete	9	3	Jay	1000.00	2026-10-09
Edit Copy Delete	10	4	Krish	3500.00	2026-10-10
Edit Copy Delete	11	4	Pooja	2800.00	2026-10-11
Edit Copy Delete	12	4	Nisha	1700.00	2026-10-12

```
SELECT
   restaurant_id,
   SUM(total_amount) AS total_revenue,
   AVG(total_amount) AS average_order_value
FROM orders
GROUP BY restaurant_id;
```

	restaurant_id	total_revenue	average_order_value
Edit Copy Delete	1	5500.00	1833.333333
Edit Copy Delete	2	5700.00	1900.000000
Edit Copy Delete	3	6500.00	2166.666667
Edit Copy Delete	4	8000.00	2666.666667

Check all *With selected:* Edit Copy Delete Export

```
SELECT
   restaurant_id,
   SUM(total_amount) AS total_revenue
FROM orders
```

```
GROUP BY restaurant_id
HAVING SUM(total_amount) > 5000;
```



	restaurant_id	total_revenue
Edit Copy Delete	1	5500.00
Edit Copy Delete	2	5700.00
Edit Copy Delete	3	6500.00
Edit Copy Delete	4	8000.00

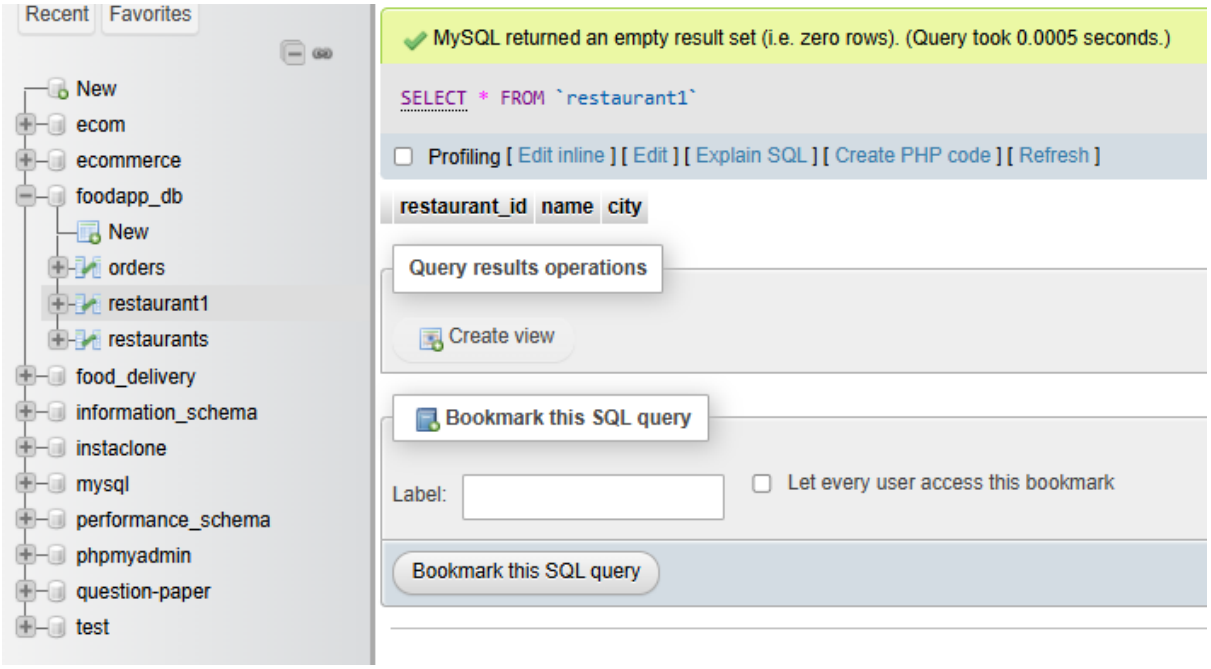
Check all *With selected:* Edit Copy Delete Export

```
SELECT *
FROM orders
ORDER BY order_date DESC
LIMIT 5;
```

	order_id	restaurant_id	customer_name	total_amount	order_date
Edit Copy Delete	12	4	Nisha	1700.00	2026-10-12
Edit Copy Delete	11	4	Pooja	2800.00	2026-10-11
Edit Copy Delete	10	4	Krish	3500.00	2026-10-10
Edit Copy Delete	9	3	Jay	1000.00	2026-10-09
Edit Copy Delete	8	3	Meera	2500.00	2026-10-08

Task 3: Multi-Table JOINs and Menu Summary View

```
CREATE TABLE restaurant1 (
   restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   city VARCHAR(50)
);
```



INSERT INTO restaurant1 (name, city)

VALUES

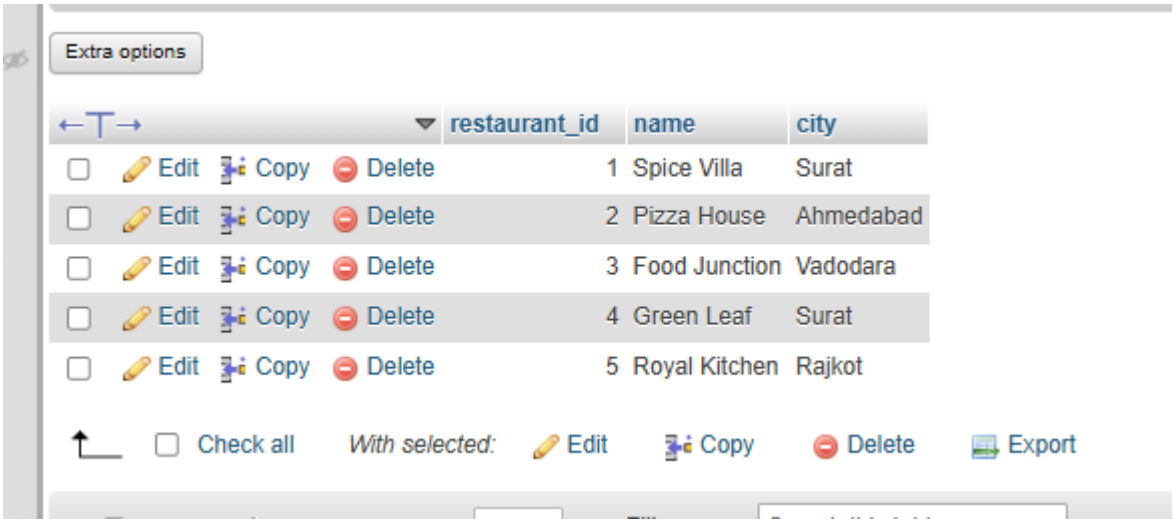
('Spice Villa', 'Surat'),

('Pizza House', 'Ahmedabad'),

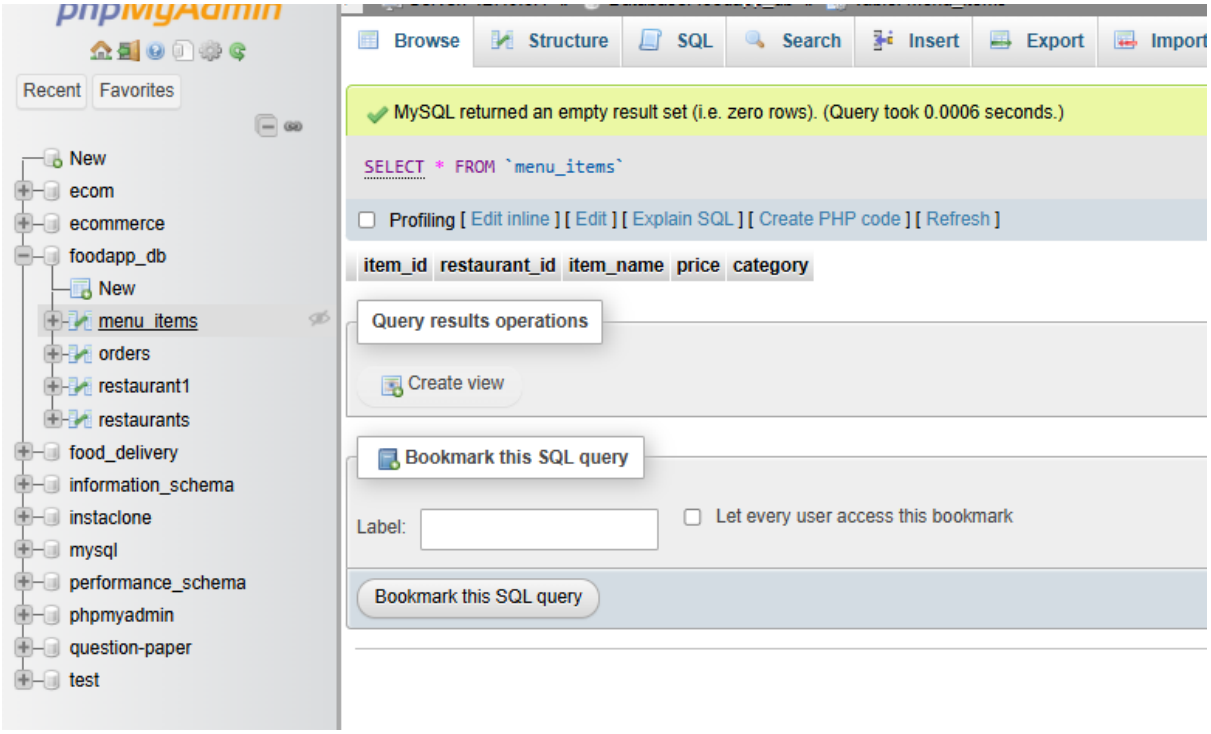
('Food Junction', 'Vadodara'),

('Green Leaf', 'Surat'),

('Royal Kitchen', 'Rajkot');



```
CREATE TABLE menu_items (
   item_id INT AUTO_INCREMENT PRIMARY KEY,
   restaurant_id INT,
   item_name VARCHAR(100),
   price DECIMAL(10,2),
   category VARCHAR(50),
   FOREIGN KEY (restaurant_id)
      REFERENCES restaurant1(restaurant_id)
);
```




```
INSERT INTO menu_items
(restaurant_id, item_name, price, category)
VALUES
(1, 'Paneer Tikka', 250, 'Starter'),
(1, 'Gujarati Thali', 180, 'Main Course'),
(1, 'Veg Biryani', 220, 'Main Course'),
```

(2, 'Margherita Pizza', 299, 'Pizza'),

(2, 'Farmhouse Pizza', 399, 'Pizza'),

(2, 'Garlic Bread', 149, 'Starter'),

(3, 'Hakka Noodles', 190, 'Chinese'),

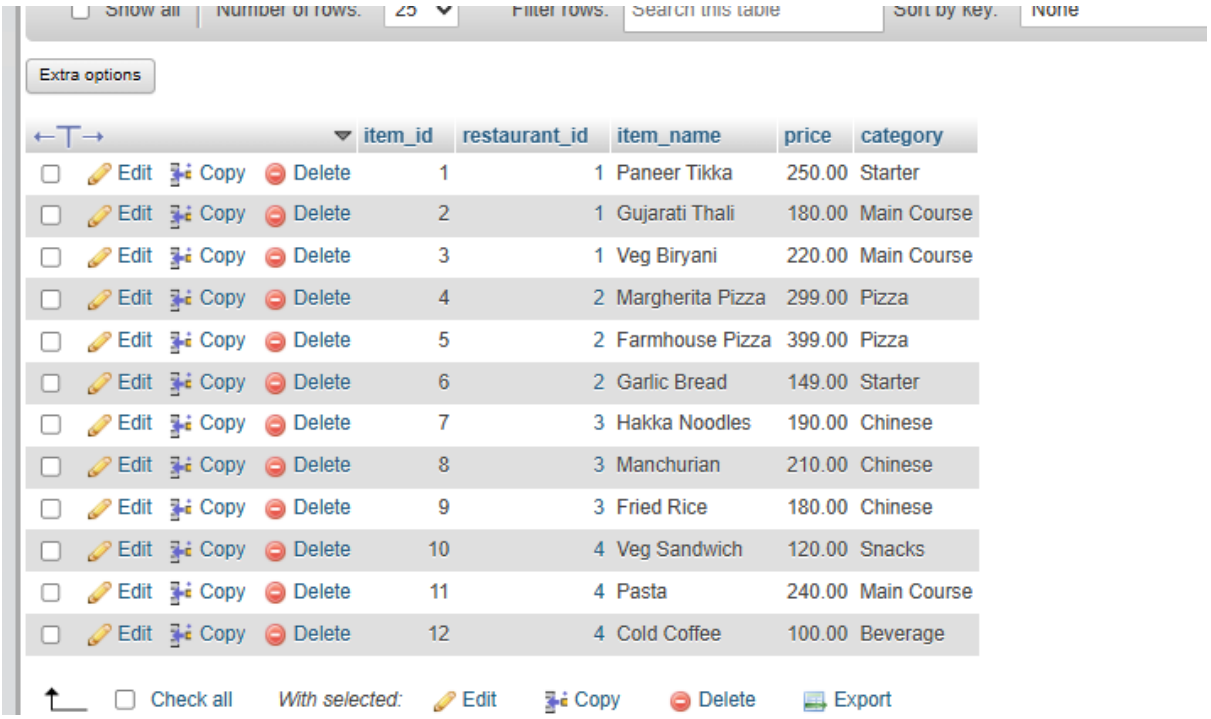
(3, 'Manchurian', 210, 'Chinese'),

(3, 'Fried Rice', 180, 'Chinese'),

(4, 'Veg Sandwich', 120, 'Snacks'),

(4, 'Pasta', 240, 'Main Course'),

(4, 'Cold Coffee', 100, 'Beverage');

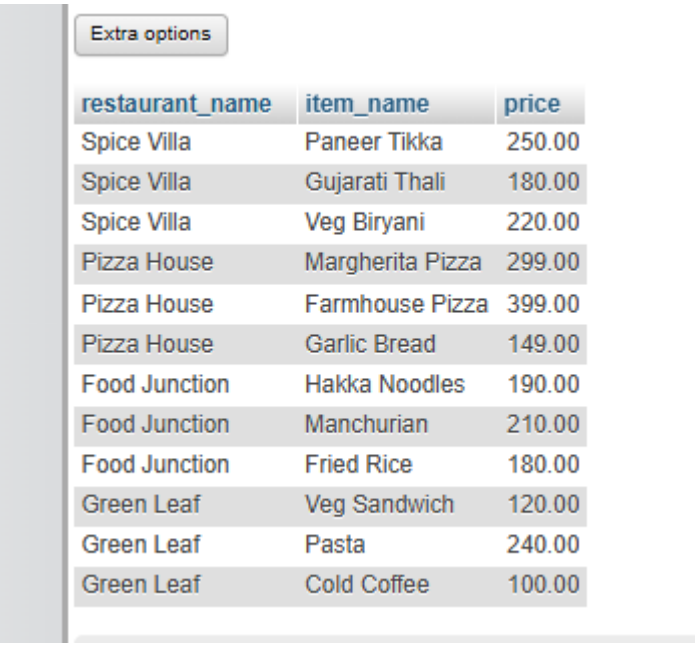


Extra options

	item_id	restaurant_id	item_name	price	category
Edit Copy Delete	1	1	Paneer Tikka	250.00	Starter
Edit Copy Delete	2	1	Gujarati Thali	180.00	Main Course
Edit Copy Delete	3	1	Veg Biryani	220.00	Main Course
Edit Copy Delete	4	2	Margherita Pizza	299.00	Pizza
Edit Copy Delete	5	2	Farmhouse Pizza	399.00	Pizza
Edit Copy Delete	6	2	Garlic Bread	149.00	Starter
Edit Copy Delete	7	3	Hakka Noodles	190.00	Chinese
Edit Copy Delete	8	3	Manchurian	210.00	Chinese
Edit Copy Delete	9	3	Fried Rice	180.00	Chinese
Edit Copy Delete	10	4	Veg Sandwich	120.00	Snacks
Edit Copy Delete	11	4	Pasta	240.00	Main Course
Edit Copy Delete	12	4	Cold Coffee	100.00	Beverage

Check all *With selected:* Edit Copy Delete Export

```
SELECT r.name AS restaurant_name,
    m.item_name,
    m.price
FROM restaurant1 r
INNER JOIN menu_items m
ON r.restaurant_id = m.restaurant_id;
```



restaurant_name	item_name	price
Spice Villa	Paneer Tikka	250.00
Spice Villa	Gujarati Thali	180.00
Spice Villa	Veg Biryani	220.00
Pizza House	Margherita Pizza	299.00
Pizza House	Farmhouse Pizza	399.00
Pizza House	Garlic Bread	149.00
Food Junction	Hakka Noodles	190.00
Food Junction	Manchurian	210.00
Food Junction	Fried Rice	180.00
Green Leaf	Veg Sandwich	120.00
Green Leaf	Pasta	240.00
Green Leaf	Cold Coffee	100.00

SELECT r.name AS restaurant_name,

m.item_name,

m.price

FROM restaurant1 r

LEFT JOIN menu_items m

ON r.restaurant_id = m.restaurant_id;

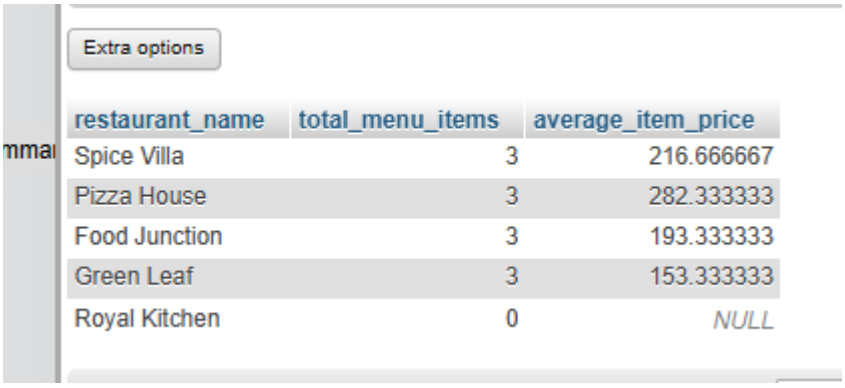


restaurant_name	item_name	price
Spice Villa	Paneer Tikka	250.00
Spice Villa	Gujarati Thali	180.00
Spice Villa	Veg Biryani	220.00
Pizza House	Margherita Pizza	299.00
Pizza House	Farmhouse Pizza	399.00
Pizza House	Garlic Bread	149.00
Food Junction	Hakka Noodles	190.00
Food Junction	Manchurian	210.00
Food Junction	Fried Rice	180.00
Green Leaf	Veg Sandwich	120.00
Green Leaf	Pasta	240.00
Green Leaf	Cold Coffee	100.00
Royal Kitchen	*NULL*	*NULL*

```
CREATE VIEW restaurant_menu_summary AS
SELECT r.name AS restaurant_name,
    COUNT(m.item_id) AS total_menu_items,
    AVG(m.price) AS average_item_price
FROM restaurant1 r
LEFT JOIN menu_items m
ON r.restaurant_id = m.restaurant_id
GROUP BY r.restaurant_id, r.name;
```




```
SELECT * FROM restaurant_menu_summary;
```

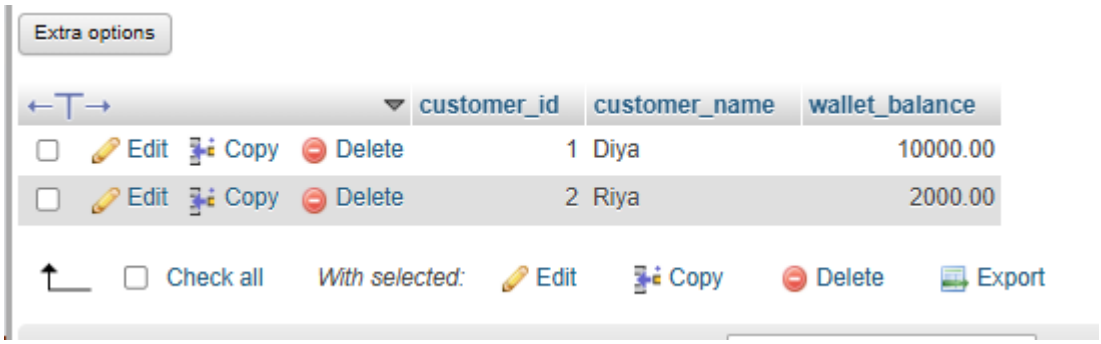


Task 4: PL/SQL Order Placement Procedure

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100),
```

```
    wallet_balance DECIMAL(10,2)
);

INSERT INTO customers
(customer_id, customer_name, wallet_balance)
VALUES
(1, 'Diya', 10000.00),
(2, 'Riya', 2000.00);
```




```
CREATE TABLE order1 (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    restaurant_id INT,
    amount DECIMAL(10,2),
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);
CREATE TABLE orders_task4 (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    restaurant_id INT,
    amount DECIMAL(10,2),
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id)
```

```
    REFERENCES customers(customer_id)
);



DELIMITER //

CREATE PROCEDURE place_order(
  IN p_customer_id INT,
  IN p_restaurant_id INT,
  IN p_amount DECIMAL(10,2)
)
BEGIN
  DECLARE v_balance DECIMAL(10,2);

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;

  SELECT wallet_balance
  INTO v_balance
  FROM customers
  WHERE customer_id = p_customer_id
  FOR UPDATE;

  IF v_balance < p_amount OR p_amount <= 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Insufficient balance or invalid amount';
```

```
    END IF;

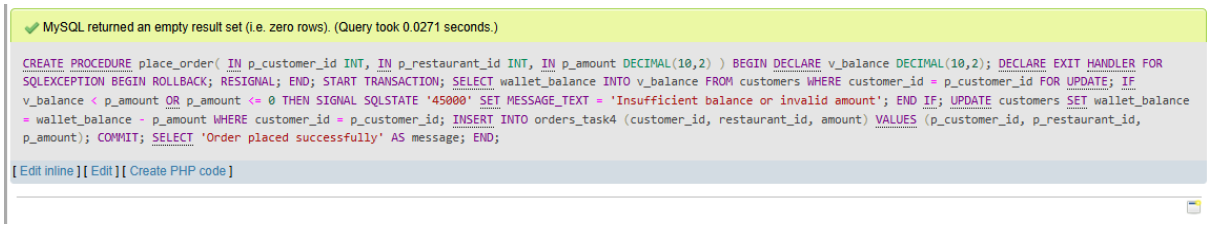
    UPDATE customers
    SET wallet_balance = wallet_balance - p_amount
    WHERE customer_id = p_customer_id;

    INSERT INTO orders_task4
        (customer_id, restaurant_id, amount)
    VALUES
        (p_customer_id, p_restaurant_id, p_amount);

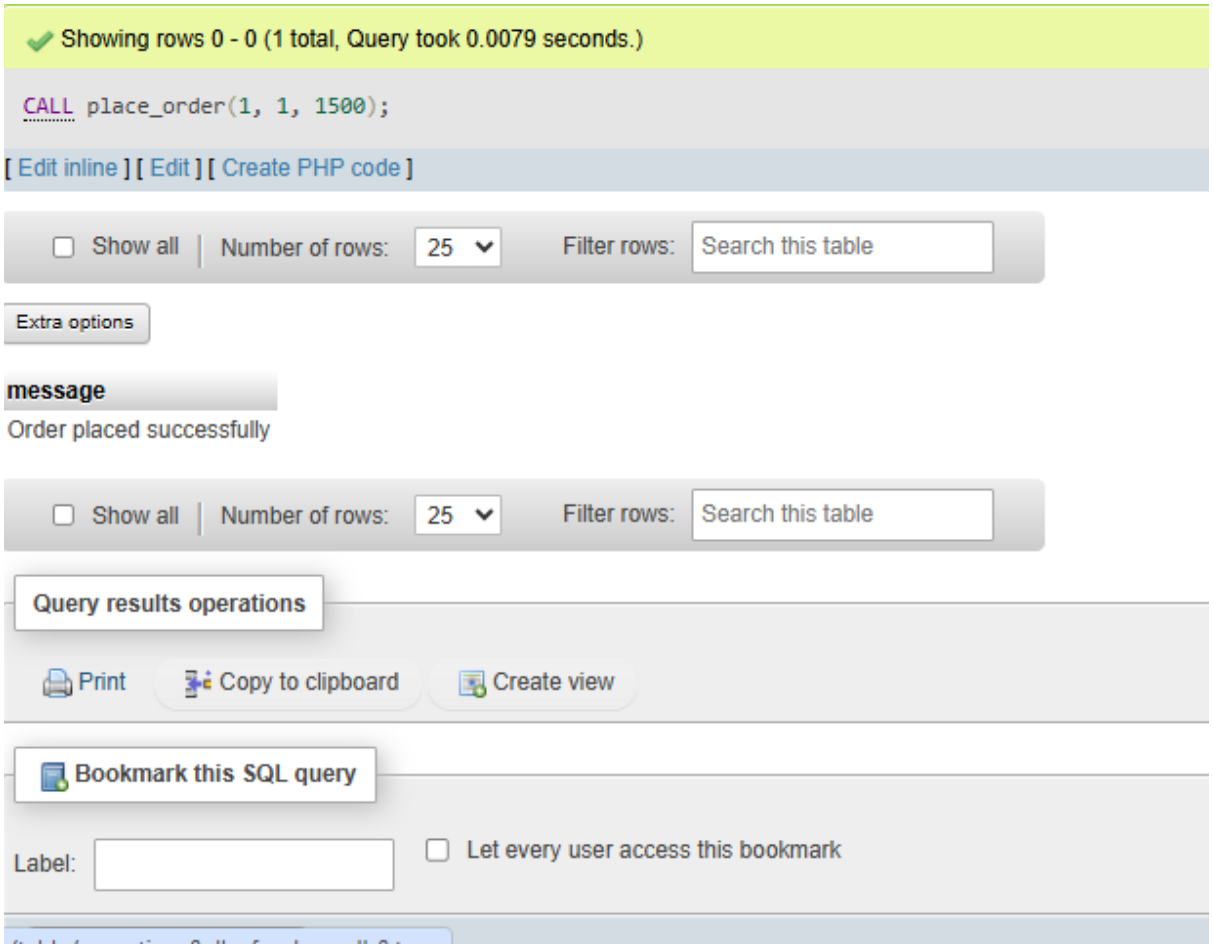
    COMMIT;

    SELECT 'Order placed successfully' AS message;
END //

DELIMITER ;
```

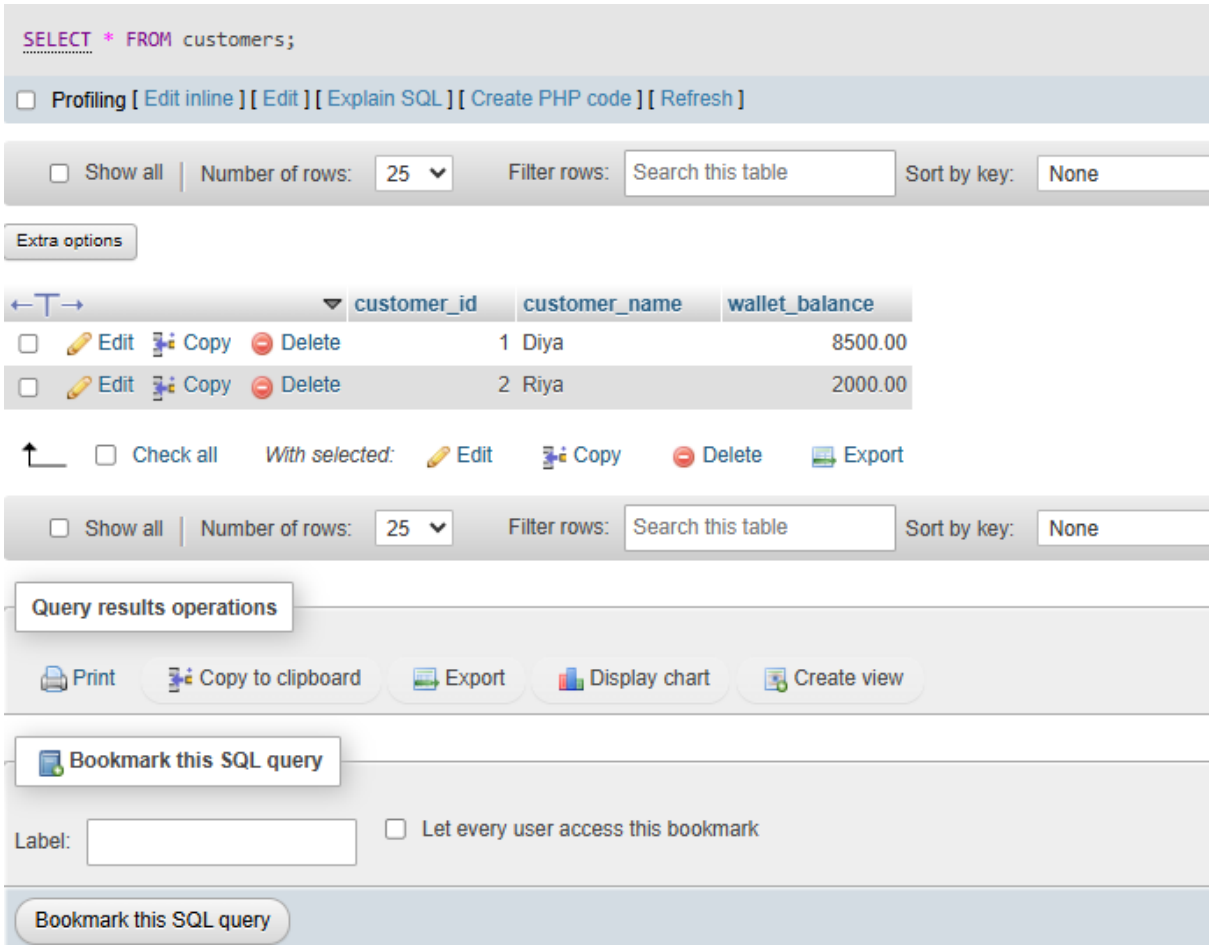



```
CALL place_order(1, 1, 1500);
```

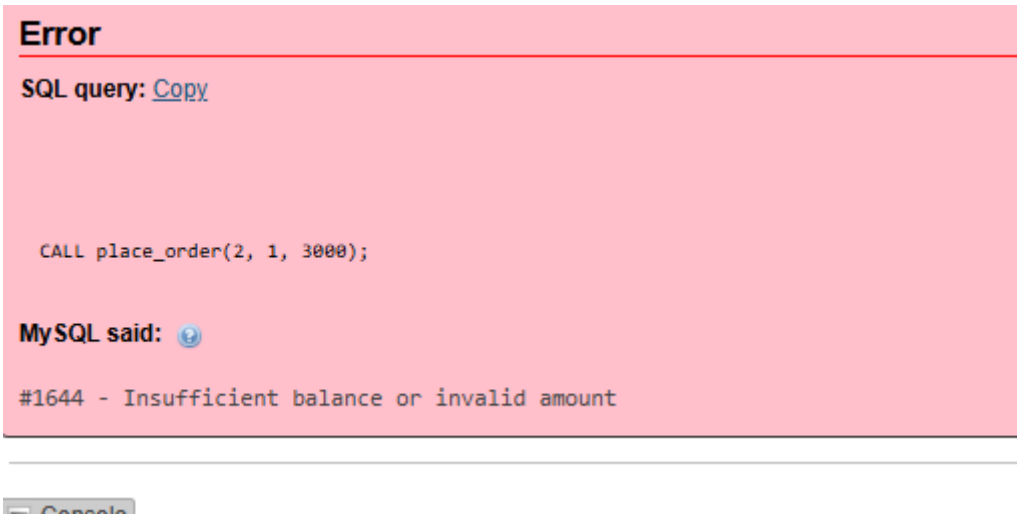


SELECT * FROM customers;

SELECT * FROM orders_task4;



CALL place_order(2, 1, 3000);

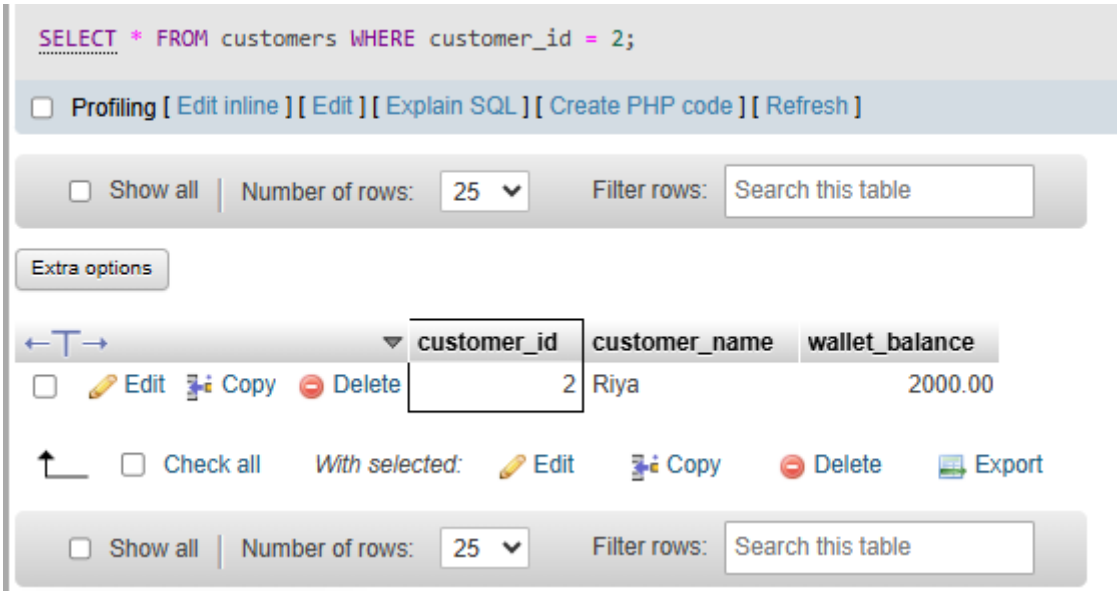


SELECT * FROM customers

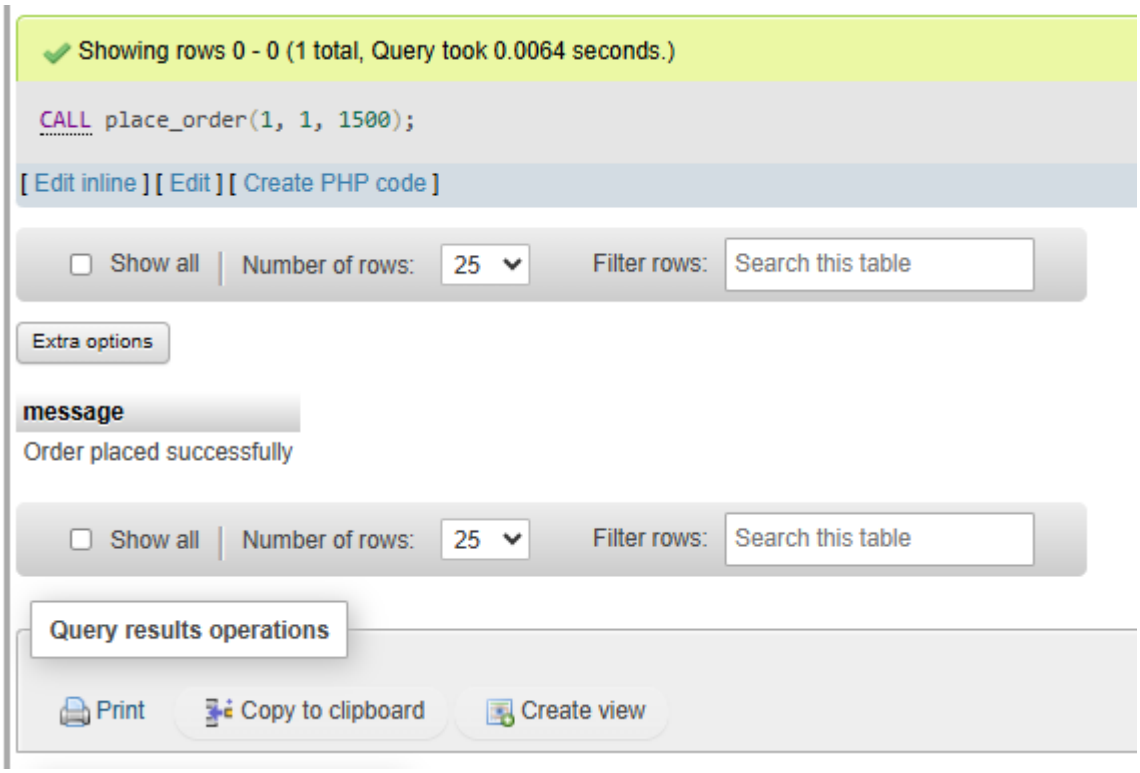
WHERE customer_id = 2;

SELECT * FROM orders_task4

WHERE customer_id = 2;



CALL place_order(1, 1, 1500);

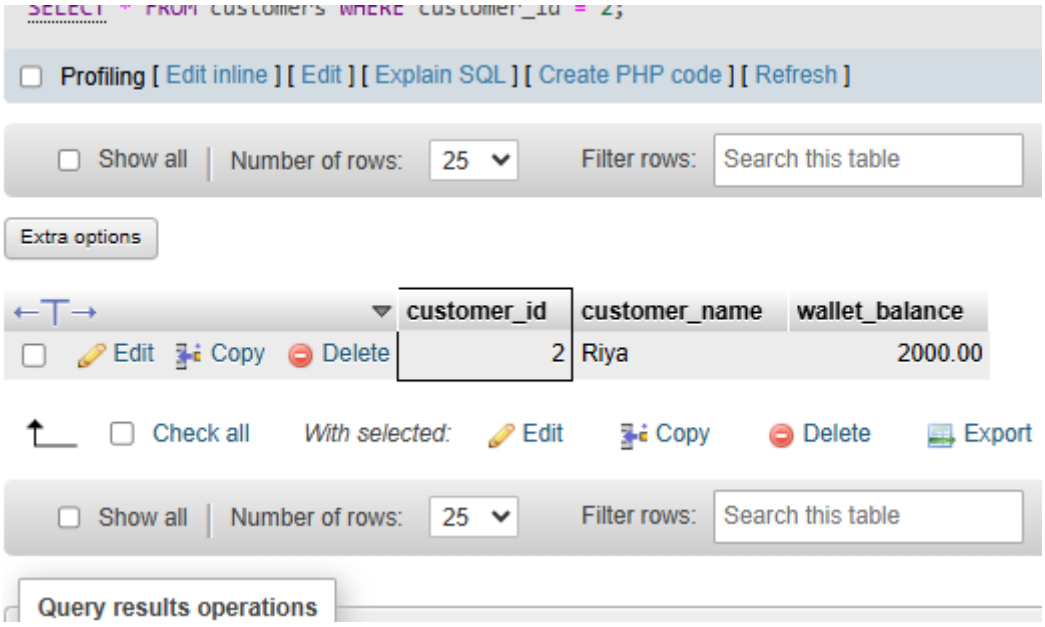


SELECT * FROM order1;

CALL place_order(2, 1, 3000);

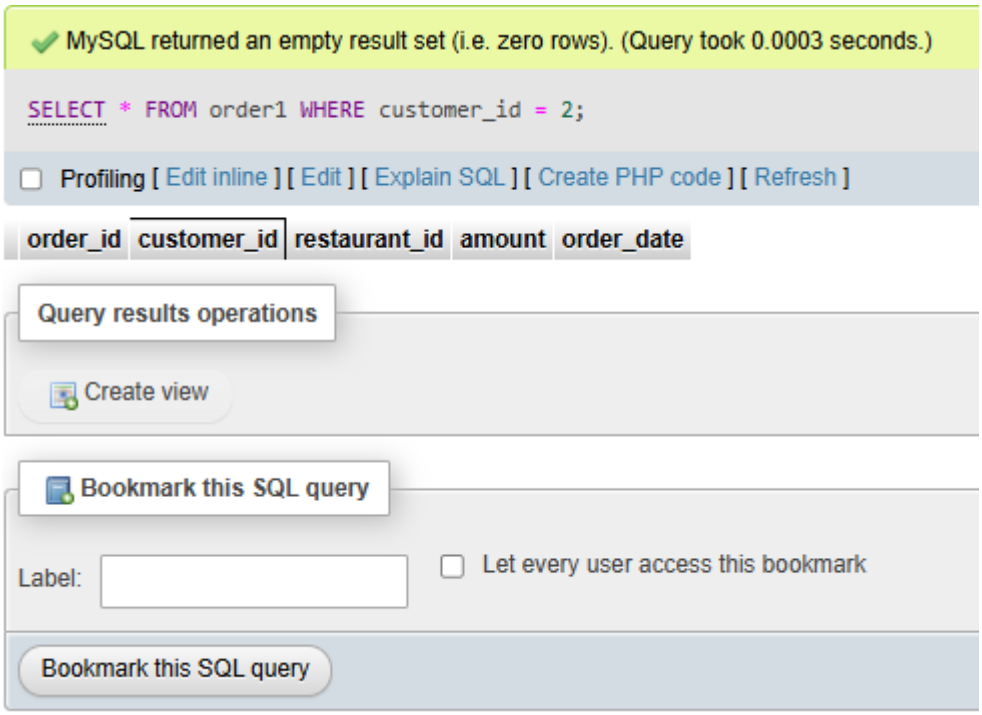
SELECT * FROM customers

WHERE customer_id = 2;



SELECT * FROM order1

WHERE customer_id = 2;



Section C — Mini Capstone Project

1. Create three tables: restaurants (restaurant_id, name, city, cuisine_type, created_at), menu_items (item_id, restaurant_id, item_name, price, category) with a FOREIGN KEY on restaurant_id, and orders (order_id, customer_name, restaurant_id, item_id, quantity, total_amount, order_date) with a FOREIGN KEY on restaurant_id — all with appropriate PRIMARY KEY and NOT NULL constraints.

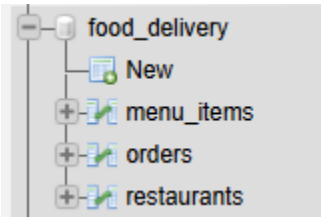
Ans :

```
CREATE DATABASE Food_Delivery;
USE Food_Delivery;


CREATE TABLE Restaurants (
   Restaurant_Id INT PRIMARY KEY AUTO_INCREMENT,
   Name VARCHAR(100) NOT NULL,
   City VARCHAR(100) NOT NULL,
   Cuisine_Type VARCHAR(100) NOT NULL,
   Created_At DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);


CREATE TABLE Menu_Items (
   Item_Id INT PRIMARY KEY AUTO_INCREMENT,
   Restaurant_Id INT NOT NULL,
   Item_Name VARCHAR(100) NOT NULL,
   Price DECIMAL(10,2) NOT NULL,
   Category VARCHAR(50) NOT NULL,
   FOREIGN KEY (Restaurant_Id)
      REFERENCES Restaurants(Restaurant_Id)
);
```

```
CREATE TABLE Orders (
   Order_Id INT PRIMARY KEY AUTO_INCREMENT,
   Customer_Name VARCHAR(100) NOT NULL,
   Restaurant_Id INT NOT NULL,
   Item_Id INT NOT NULL,
   Quantity INT NOT NULL,
   Total_Amount DECIMAL(10,2) NOT NULL,
   Order_Date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
   FOREIGN KEY (Restaurant_Id)
      REFERENCES Restaurants(Restaurant_Id)
);
```



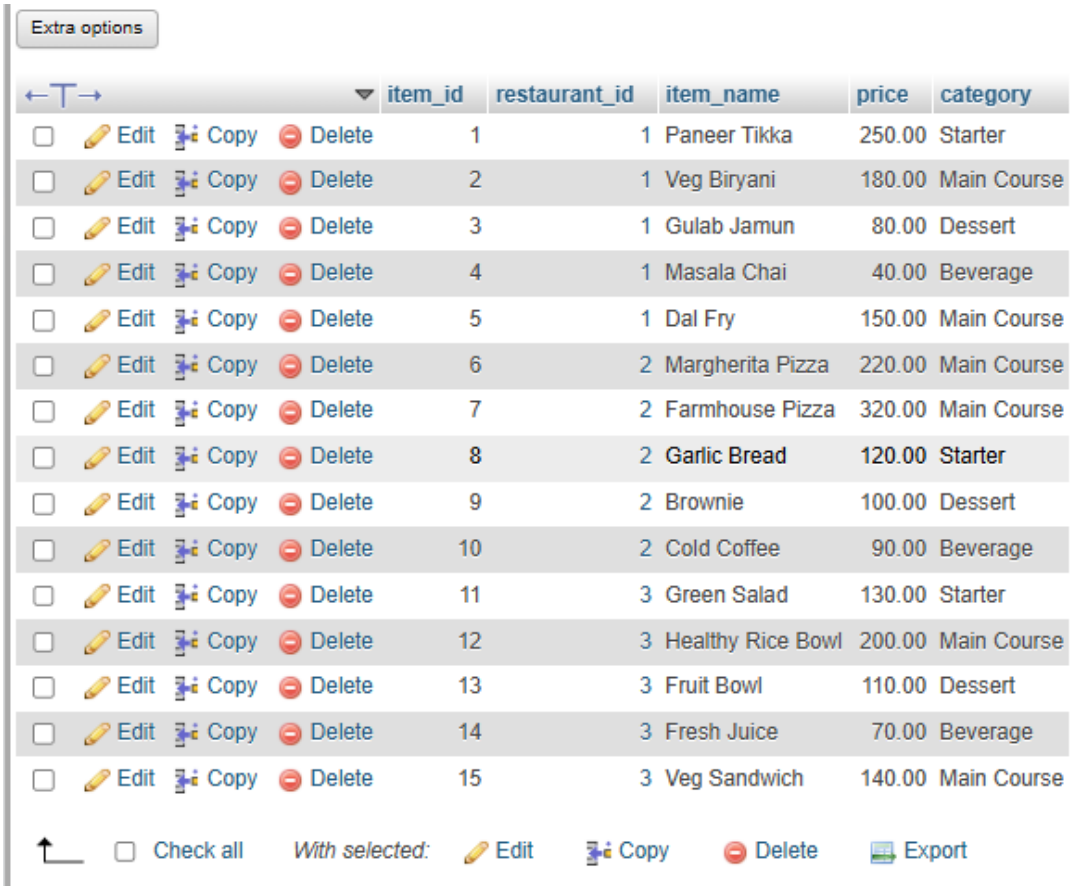
2.Populate with at least 3 restaurants, 4 menu categories, and 15 order records distributed across restaurants.

```
INSERT INTO restaurants (name, city, cuisine_type)
VALUES
('Spice Villa', 'Surat', 'Indian'),
('Pizza Point', 'Surat', 'Italian'),
('Green Bowl', 'Ahmedabad', 'Healthy Food');
```

	restaurant_id	name	city	cuisine_type	created_at
Edit Copy Delete	1	Spice Villa	Surat	Indian	2026-10-12 22:06:20
Edit Copy Delete	2	Pizza Point	Surat	Italian	2026-10-12 22:06:20
Edit Copy Delete	3	Green Bowl	Ahmedabad	Healthy Food	2026-10-12 22:06:20

```
INSERT INTO menu_items
(restaurant_id, item_name, price, category)
```

VALUES
(1, 'Paneer Tikka', 250, 'Starter'),
(1, 'Veg Biryani', 180, 'Main Course'),
(1, 'Gulab Jamun', 80, 'Dessert'),
(1, 'Masala Chai', 40, 'Beverage'),
(1, 'Dal Fry', 150, 'Main Course'),
(2, 'Margherita Pizza', 220, 'Main Course'),
(2, 'Farmhouse Pizza', 320, 'Main Course'),
(2, 'Garlic Bread', 120, 'Starter'),
(2, 'Brownie', 100, 'Dessert'),
(2, 'Cold Coffee', 90, 'Beverage'),
(3, 'Green Salad', 130, 'Starter'),
(3, 'Healthy Rice Bowl', 200, 'Main Course'),
(3, 'Fruit Bowl', 110, 'Dessert'),
(3, 'Fresh Juice', 70, 'Beverage'),
(3, 'Veg Sandwich', 140, 'Main Course');

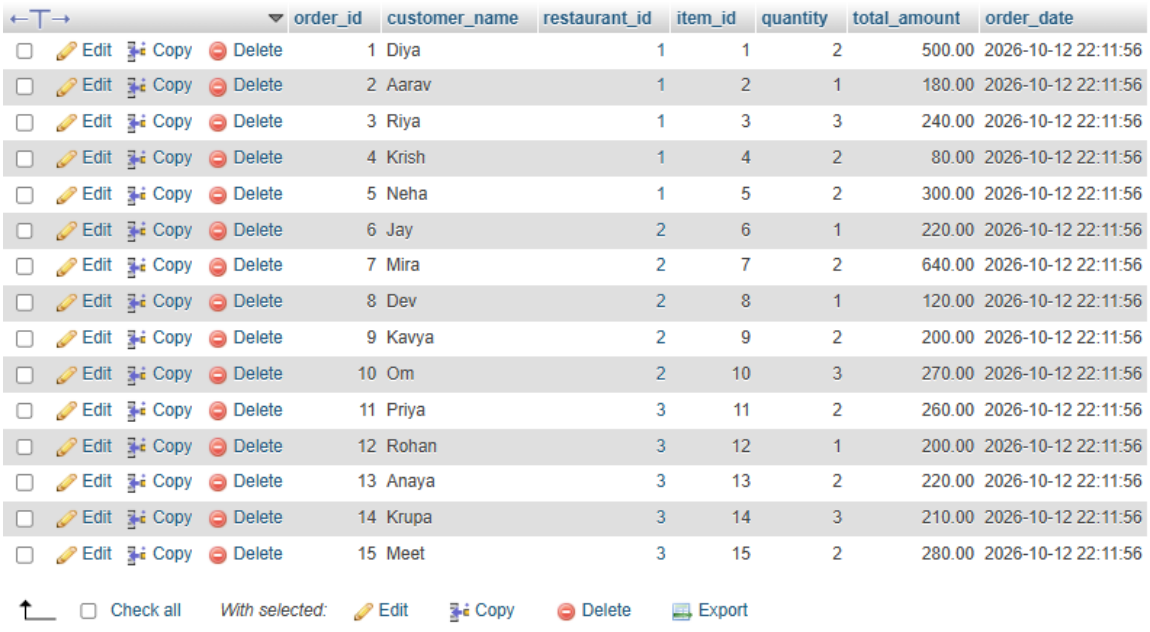

Extra options

	item_id	restaurant_id	item_name	price	category
Edit Copy Delete	1	1	Paneer Tikka	250.00	Starter
Edit Copy Delete	2	1	Veg Biryani	180.00	Main Course
Edit Copy Delete	3	1	Gulab Jamun	80.00	Dessert
Edit Copy Delete	4	1	Masala Chai	40.00	Beverage
Edit Copy Delete	5	1	Dal Fry	150.00	Main Course
Edit Copy Delete	6	2	Margherita Pizza	220.00	Main Course
Edit Copy Delete	7	2	Farmhouse Pizza	320.00	Main Course
Edit Copy Delete	8	2	Garlic Bread	120.00	Starter
Edit Copy Delete	9	2	Brownie	100.00	Dessert
Edit Copy Delete	10	2	Cold Coffee	90.00	Beverage
Edit Copy Delete	11	3	Green Salad	130.00	Starter
Edit Copy Delete	12	3	Healthy Rice Bowl	200.00	Main Course
Edit Copy Delete	13	3	Fruit Bowl	110.00	Dessert
Edit Copy Delete	14	3	Fresh Juice	70.00	Beverage
Edit Copy Delete	15	3	Veg Sandwich	140.00	Main Course

Check all With selected: Edit Copy Delete Export

INSERT INTO orders
(customer_name, restaurant_id, item_id, quantity, total_amount)
VALUES

('Diya', 1, 1, 2, 500),
('Aarav', 1, 2, 1, 180),
('Riya', 1, 3, 3, 240),
('Krish', 1, 4, 2, 80),
('Neha', 1, 5, 2, 300),
('Jay', 2, 6, 1, 220),
('Mira', 2, 7, 2, 640),
('Dev', 2, 8, 1, 120),
('Kavya', 2, 9, 2, 200),
('Om', 2, 10, 3, 270),
('Priya', 3, 11, 2, 260),
('Rohan', 3, 12, 1, 200),
('Anaya', 3, 13, 2, 220),
('Krupa', 3, 14, 3, 210),
('Meet', 3, 15, 2, 280);



	order_id	customer_name	restaurant_id	item_id	quantity	total_amount	order_date
Edit Copy Delete	1	Diya	1	1	2	500.00	2026-10-12 22:11:56
Edit Copy Delete	2	Aarav	1	2	1	180.00	2026-10-12 22:11:56
Edit Copy Delete	3	Riya	1	3	3	240.00	2026-10-12 22:11:56
Edit Copy Delete	4	Krish	1	4	2	80.00	2026-10-12 22:11:56
Edit Copy Delete	5	Neha	1	5	2	300.00	2026-10-12 22:11:56
Edit Copy Delete	6	Jay	2	6	1	220.00	2026-10-12 22:11:56
Edit Copy Delete	7	Mira	2	7	2	640.00	2026-10-12 22:11:56
Edit Copy Delete	8	Dev	2	8	1	120.00	2026-10-12 22:11:56
Edit Copy Delete	9	Kavya	2	9	2	200.00	2026-10-12 22:11:56
Edit Copy Delete	10	Om	2	10	3	270.00	2026-10-12 22:11:56
Edit Copy Delete	11	Priya	3	11	2	260.00	2026-10-12 22:11:56
Edit Copy Delete	12	Rohan	3	12	1	200.00	2026-10-12 22:11:56
Edit Copy Delete	13	Anaya	3	13	2	220.00	2026-10-12 22:11:56
Edit Copy Delete	14	Krupa	3	14	3	210.00	2026-10-12 22:11:56
Edit Copy Delete	15	Meet	3	15	2	280.00	2026-10-12 22:11:56

Check all With selected: Edit Copy Delete Export

3. Create a VIEW named restaurant_sales_summary showing restaurant name, total number of orders, and total revenue grouped by restaurant.

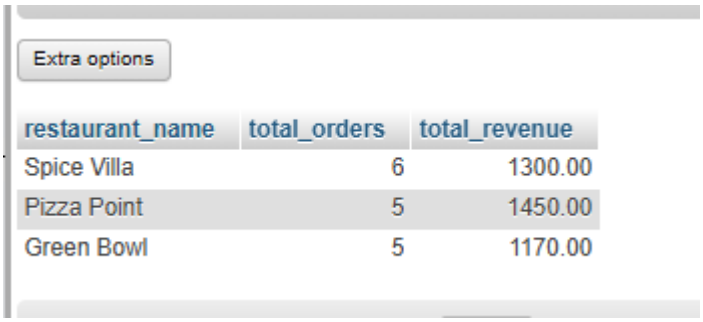
CREATE VIEW restaurant_sales_summary AS

SELECT

```
    r.Name AS restaurant_name,
    COUNT(o.Order_Id) AS total_orders,
    COALESCE(SUM(o.Total_Amount), 0) AS total_revenue
FROM Restaurants r
LEFT JOIN Orders o
    ON r.Restaurant_Id = o.Restaurant_Id
GROUP BY r.Restaurant_Id, r.Name;
```

View check :

```
SELECT * FROM restaurant_sales_summary;
```



Extra options

restaurant_name	total_orders	total_revenue
Spice Villa	6	1300.00
Pizza Point	5	1450.00
Green Bowl	5	1170.00

4. Create a stored procedure named `add_order` that accepts customer name, restaurant ID, item ID and quantity, calculates total amount using the menu item price, validates the restaurant, and rolls back if the restaurant does not exist.

```
DELIMITER //

CREATE PROCEDURE add_order(
    IN p_customer_name VARCHAR(100),
```

```
    IN p_restaurant_id INT,
    IN p_item_id INT,
    IN p_quantity INT
)
BEGIN
    DECLARE v_restaurant_count INT DEFAULT 0;
    DECLARE v_price DECIMAL(10,2);

    START TRANSACTION;

    SELECT COUNT(*)
    INTO v_restaurant_count
    FROM Restaurants
    WHERE Restaurant_Id = p_restaurant_id;

    IF v_restaurant_count = 0 THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Restaurant does not exist';
    ELSE
        SELECT Price INTO v_price
        FROM Menu_Items
        WHERE Item_Id = p_item_id
          AND Restaurant_Id = p_restaurant_id;

        INSERT INTO Orders
```

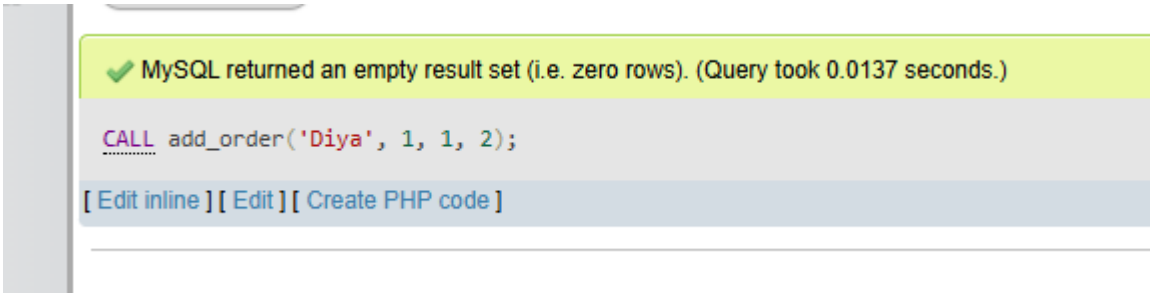
```
        (Customer_Name, Restaurant_Id, Item_Id,
         Quantity, Total_Amount, Order_Date)
        VALUES
        (p_customer_name, p_restaurant_id, p_item_id,
         p_quantity, v_price * p_quantity, NOW());

        COMMIT;
    END IF;
END //

DELIMITER ;
```

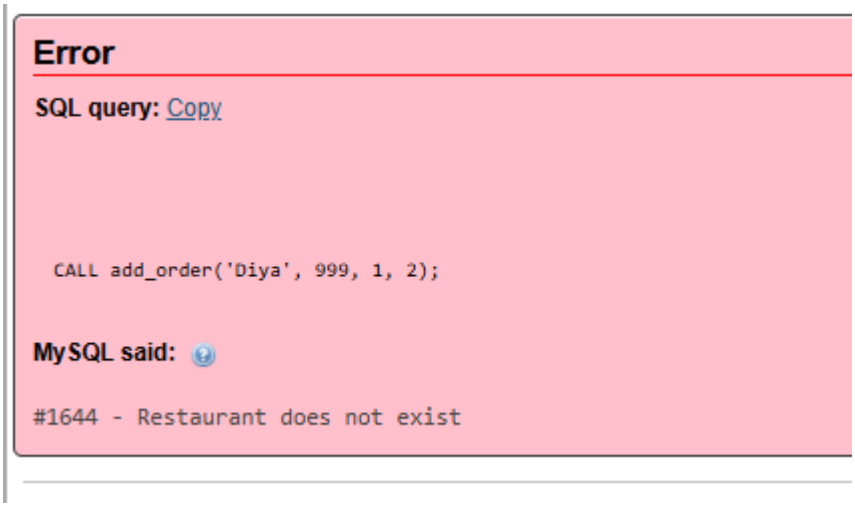
Procedure test :

```
CALL add_order('Diya', 1, 1, 2);
```



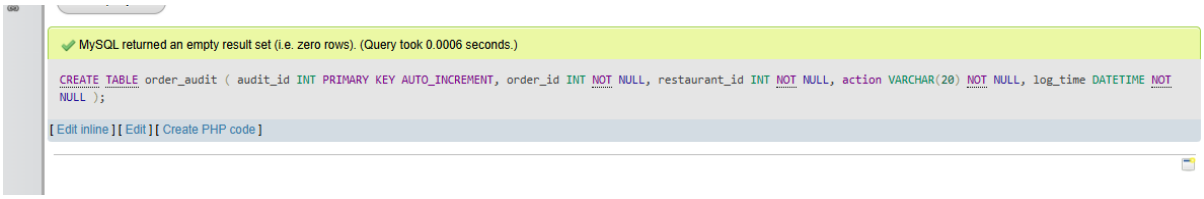
Invalide restaurant :

```
CALL add_order('Diya', 999, 1, 2);
```



5. Create an `order_audit` table and a trigger that logs every new order with order ID, restaurant ID, action `'INSERT'`, and current time.

```
CREATE TABLE order_audit (
    audit_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL,
    restaurant_id INT NOT NULL,
    action VARCHAR(20) NOT NULL,
    log_time DATETIME NOT NULL
);
```




```
DELIMITER //
```

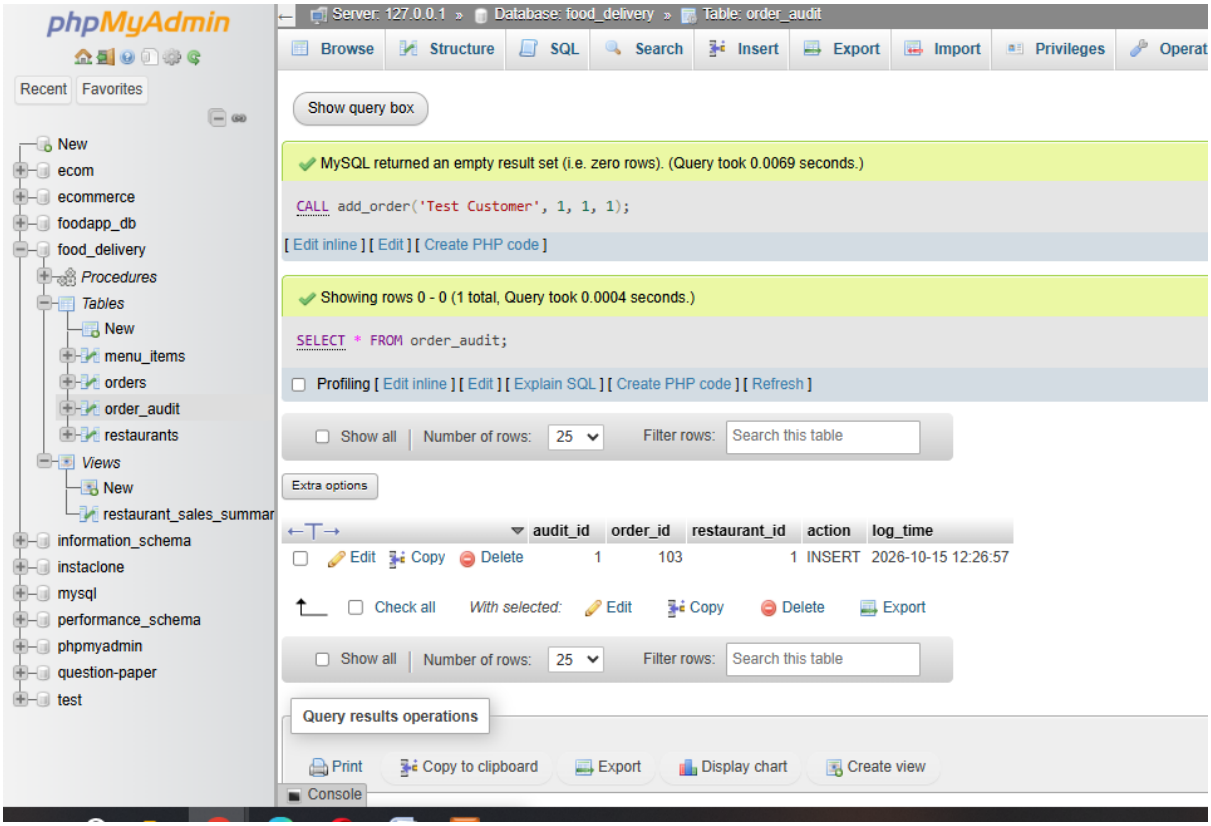
```
CREATE TRIGGER after_order_insert
AFTER INSERT ON Orders
FOR EACH ROW
BEGIN
   INSERT INTO order_audit
   (order_id, restaurant_id, action, log_time)
   VALUES
   (NEW.Order_Id, NEW.Restaurant_Id, 'INSERT', NOW());
END //

DELIMITER ;
```

Trigger test

```
CALL add_order('Test Customer', 1, 1, 1);

SELECT * FROM order_audit;
```



Session -D

Accepts a restaurant_id and a month number (1–12) as inputs and returns a summary showing total orders, total revenue, and average order value for that restaurant in that month.

Ans.

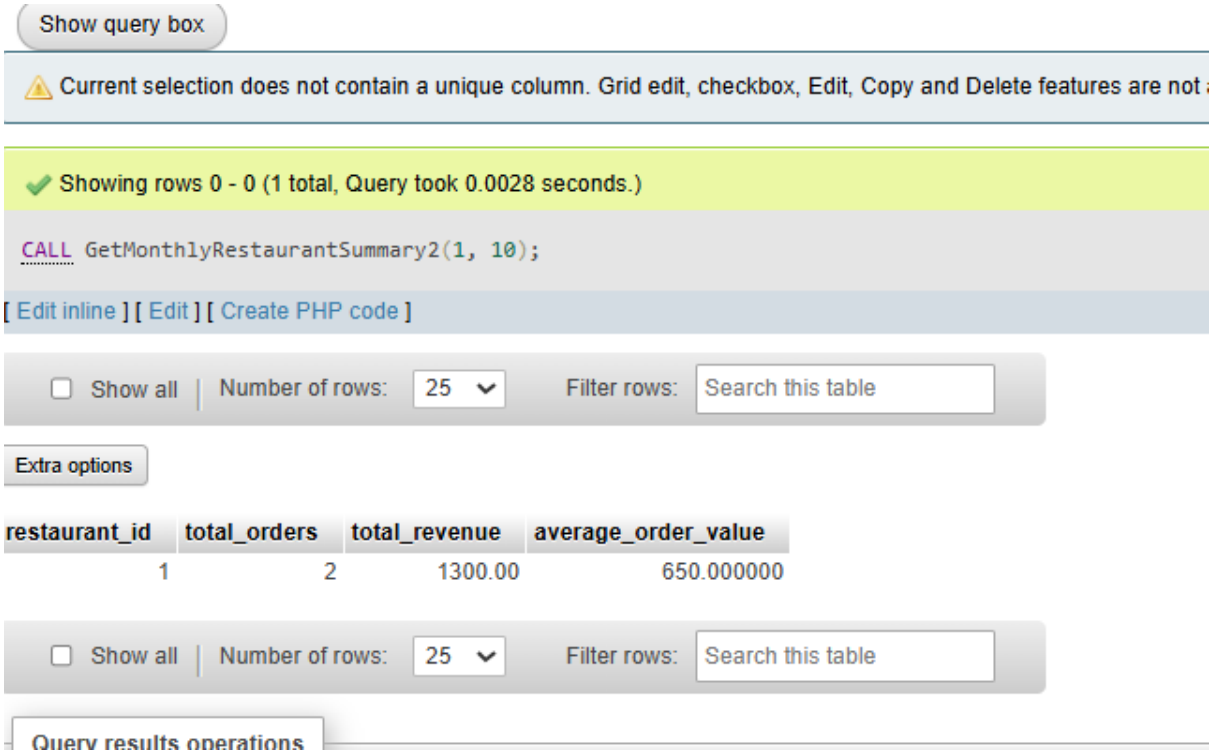
```
DELIMITER //

CREATE PROCEDURE GetRestaurantMonthlySummary2(
```

```
    IN p_restaurant_id INT,
    IN p_month INT
)
BEGIN
    SELECT
        restaurant_id,
        COUNT(order_id) AS total_orders,
        SUM(total_amount) AS total_revenue,
        AVG(total_amount) AS average_order_value
    FROM orders
    WHERE restaurant_id = p_restaurant_id
      AND MONTH(order_date) = p_month
    GROUP BY restaurant_id;
END //

DELIMITER ;

CALL GetRestaurantMonthlySummary2(1, 10);
```



2. Handles the case where the restaurant has no orders in that month by returning a meaningful message rather than an empty result.

```
DROP PROCEDURE IF EXISTS GetMonthlyRestaurantSummary;


DELIMITER //


CREATE PROCEDURE GetMonthlyRestaurantSummary(
   IN p_restaurant_id INT,
   IN p_month INT
)
BEGIN
   IF NOT EXISTS (
      SELECT 1
      FROM restaurant_orders
```

```
    WHERE restaurant_id = p_restaurant_id
      AND MONTH(order_date) = p_month
  ) THEN

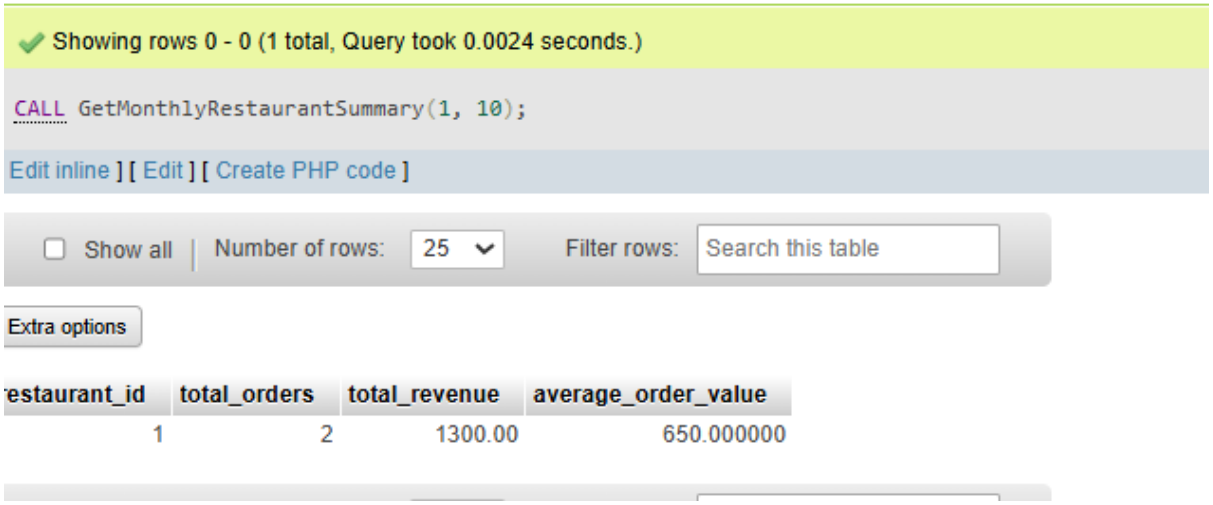
    SELECT
      'No orders found for this restaurant in the selected month.'
      AS message;

  ELSE

    SELECT
      p_restaurant_id AS restaurant_id,
      COUNT(order_id) AS total_orders,
      SUM(total_amount) AS total_revenue,
      AVG(total_amount) AS average_order_value
    FROM restaurant_orders
    WHERE restaurant_id = p_restaurant_id
      AND MONTH(order_date) = p_month;

  END IF;
END //

DELIMITER ;
CALL GetMonthlyRestaurantSummary(1, 10);
```



3.Uses a SELECT with GROUP BY and aggregate functions to compute the summary values correctly

```
DROP PROCEDURE IF EXISTS GetMonthlyRestaurantSummary;

DELIMITER //

CREATE PROCEDURE GetMonthlyRestaurantSummary(
   IN p_restaurant_id INT,
   IN p_month INT
)
BEGIN
   IF NOT EXISTS (
      SELECT 1
      FROM restaurant_orders
      WHERE restaurant_id = p_restaurant_id
        AND MONTH(order_date) = p_month
   ) THEN

      SELECT
         'No orders found for this restaurant in the selected month.'
```

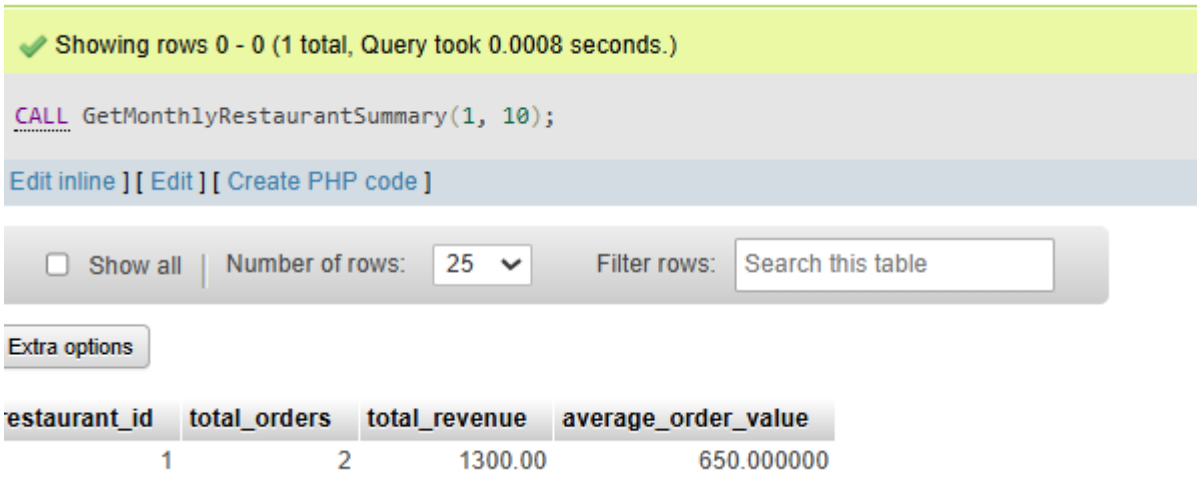
```
        AS message;

  ELSE

    SELECT
      restaurant_id,
      COUNT(order_id) AS total_orders,
      SUM(total_amount) AS total_revenue,
      AVG(total_amount) AS average_order_value
    FROM restaurant_orders
    WHERE restaurant_id = p_restaurant_id
     AND MONTH(order_date) = p_month
    GROUP BY restaurant_id;

  END IF;
END //

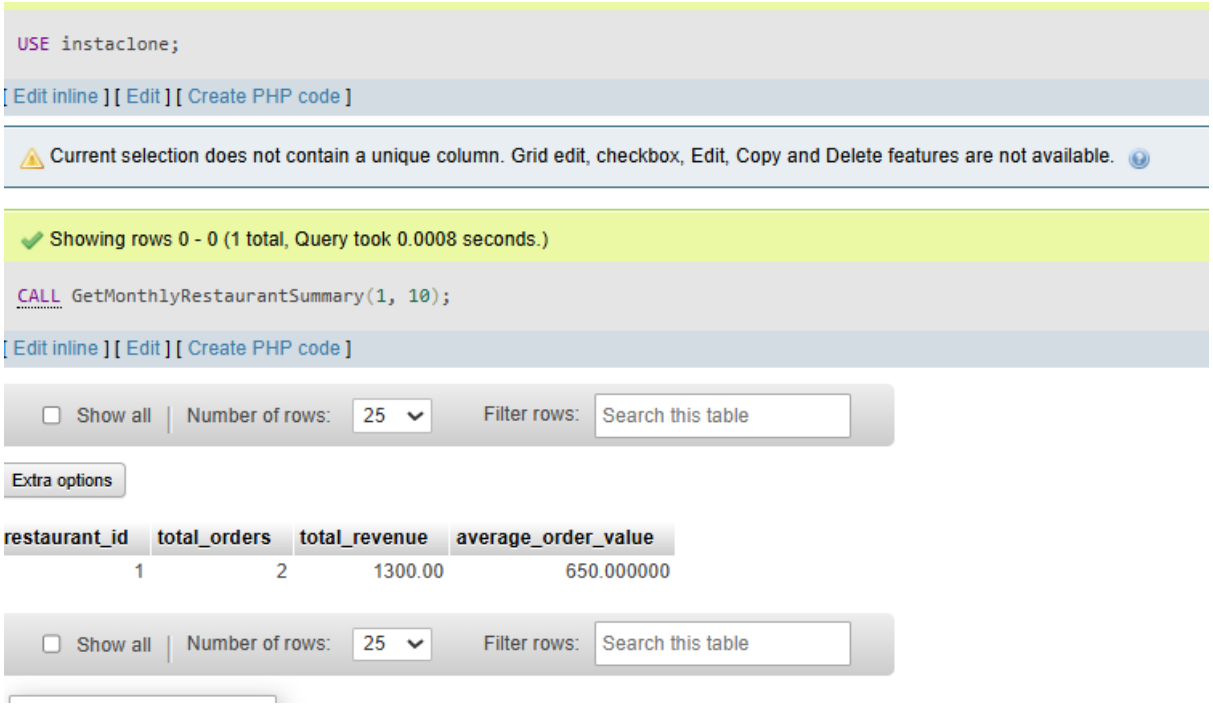
DELIMITER ;
CALL GetMonthlyRestaurantSummary(1, 10);
```



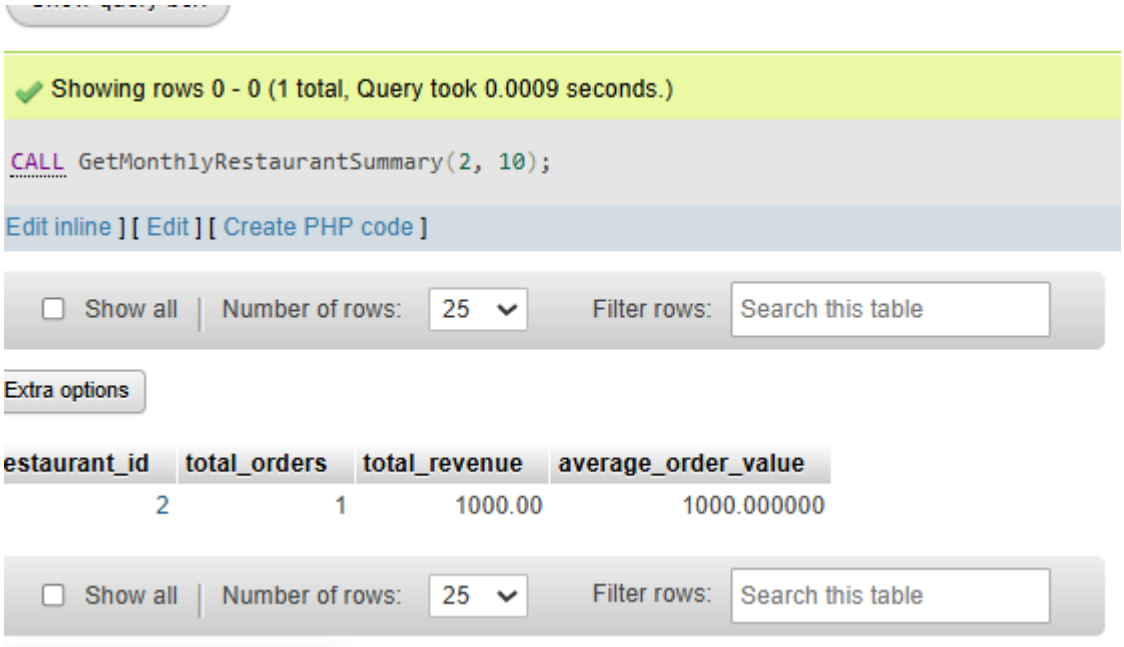
4. Runs successfully in MySQL Workbench — screenshot output for at least two different restaurant_id and month combinations.

USE instaclone;

CALL GetMonthlyRestaurantSummary(1, 10);



CALL GetMonthlyRestaurantSummary(2, 10);



Question-2

1. **Exact prompts given to AI**

   “Accepts a restaurant_id and a month number (1–12) as inputs and returns a summary showing total orders, total revenue, and average order value for that restaurant in that month.”

   “Handles the case where the restaurant has no orders in that month by returning a meaningful message rather than an empty result.”

   “Uses a SELECT with GROUP BY and aggregate functions to compute the summary values correctly.”

2. **Original and corrected stored procedure**

```
DROP PROCEDURE IF EXISTS GetMonthlyRestaurantSummary;

DELIMITER //

CREATE PROCEDURE GetMonthlyRestaurantSummary(
  IN p_restaurant_id INT,
  IN p_month INT
)
BEGIN
  IF NOT EXISTS (
    SELECT 1
    FROM restaurant_orders
    WHERE restaurant_id = p_restaurant_id
     AND MONTH(order_date) = p_month
  ) THEN
    SELECT
      'No orders found for this restaurant in the selected month.'
      AS message;
  ELSE
    SELECT
      restaurant_id,
      COUNT(order_id) AS total_orders,
      SUM(total_amount) AS total_revenue,
      AVG(total_amount) AS average_order_value
    FROM restaurant_orders
    WHERE restaurant_id = p_restaurant_id
     AND MONTH(order_date) = p_month
    GROUP BY restaurant_id;
  END IF;
END //
```

DELIMITER ;

Improved version: One important limitation is that the original procedure does not validate the month input. Values such as 0 or 13 are accepted, but they are not valid months. The improved version rejects invalid months before running the summary.

```
DROP PROCEDURE IF EXISTS GetMonthlyRestaurantSummary;

DELIMITER //

CREATE PROCEDURE GetMonthlyRestaurantSummary(
  IN p_restaurant_id INT,
  IN p_month INT
)
BEGIN
  IF p_month IS NULL OR p_month < 1 OR p_month > 12 THEN
    SELECT 'Invalid month. Enter a month from 1 to 12.'
    AS message;

  ELSEIF NOT EXISTS (
    SELECT 1
    FROM restaurant_orders
    WHERE restaurant_id = p_restaurant_id
     AND MONTH(order_date) = p_month
  ) THEN
    SELECT
      'No orders found for this restaurant in the selected month.'
      AS message;

  ELSE
    SELECT
      restaurant_id,
      COUNT(order_id) AS total_orders,
      COALESCE(SUM(total_amount), 0) AS total_revenue,
      COALESCE(AVG(total_amount), 0) AS average_order_value
    FROM restaurant_orders
    WHERE restaurant_id = p_restaurant_id
     AND MONTH(order_date) = p_month
    GROUP BY restaurant_id;
  END IF;
END //
```

DELIMITER ;

3. Three-to-four-line explanation

The AI-generated procedure did not validate the month input, so invalid values such as 0 or 13 were accepted without a clear error message. I improved the procedure by adding validation to allow only months from 1 to 12. I also retained the no-orders message and used COALESCE with the aggregate totals to handle NULL values safely. This makes the procedure more reliable and user-friendly.

CALL GetMonthlyRestaurantSummary(1, 10);

CALL GetMonthlyRestaurantSummary(3, 9);

CALL GetMonthlyRestaurantSummary(1, 13);

